Typische Komplexitätsfunktionen

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Effizienzvergleich von Algorithmen
- Asymptotische Analyse

Was wird in diesem Foliensatz besprochen?

- Typische Komplexitätsfunktionen
- Konkrete Analysebeispiele

Was lernen Sie kennen?

- Komplexitätsfunktionen
- Vorgehensweise bei der Analyse von Algorithmen

Beispiele für Komplexitätsfunktionen

	Bezeichnung
$O(1)$	Konstant
$O(\log n)$	Logarithmisch
$O(n)$	Linear
$O(n \log n)$	Log-Linear
$O(n^2)$	Quadratisch
$O(n^3)$	Kubisch
$O(k^n)$	Exponentiell

Konstant – O(1)

Unabhängig von der Eingabegröße *n* ist der Aufwand für eine Operation konstant

Konstant auf einem bestimmten System

- Aber unterschiedlich (konstant) bei verschiedenen Systemen

Beispiel: Bestimmen der Länge eines Arrays

- Wenn die Länge beim Array gespeichert wird (wie z. B. in Java)

Logarithmisch – O(log n)

Potenz x^n mit Schleifen berechnen
• x und n sind positive Zahlen • Einfache Variante multipliziert n-mal x in einer Schleife • Effizientere Version (siehe Beispiel)

```
private static long pow(long x, long n) {
    long result = 1;
    while (n > 0) {
        if (n % 2 == 1) {
            result = result * x;
        }
        x = x * x;
        n = n / 2;
    }
    return result;
}
```

Entscheidender Bereich (abhängig von der Größe von n)
Um x^n zu berechnen sind bei $n \geq 1$ $\lfloor \log_2(n) + 1 \rfloor$ Schleifendurchläufe notwendig, daher $O(\log n)$.

Logarithmisch – Hinweis

Logarithmen zu verschiedenen Basen unterscheiden sich nur um einen konstanten Faktor voneinander

Daher kann die Basis weggelassen werden

Linear – O(n)

Laufzeit wächst linear mit der Eingabegröße

Beispiel (Summe aller Zahlen eines Arrays bestimmen)

```
private static int addNumbers(int[] input) {
    int sum = 0;
    for (int i = 0; i < input.length; i++) {
        sum += input[i];
    }
    return sum;
}
```

Jedes Element muss betrachtet werden

Linear – Rekursionsbeispiel

```
// n >= 0
private static long factorial(int n) {
    return n <= 0 ? 1 : n * factorial(n - 1);
}
```

Lineare Abfolge von Aufrufen

factorial(n) -> factorial(n-1) -> factorial(n-2) -> … -> factorial(1) -> factorial(0)

Speicherbedarf (Frames) wächst aber auch linear mit der Eingabegröße

Log-Linear – O(n log n)

Eine wichtige Klasse mit vielen Algorithmen

Beispiele

- Schnelle Sortieralgorithmen
- Werden noch ausführlich besprochen

Quadratisch – O(n²)

```java
private static boolean allEntriesDifferent(int data[]) {
    for (int i = 0; i < data.length; i++) {
        for (int j = i + 1; j < data.length; j++) {
            if (data[i] == data[j]) {
                return false;
            }
        }
    }
    return true;
}
```

Überprüft, ob alle Elemente im Array unterschiedlich sind. Äußere Schleife abhängig von der Länge von data.

Innere Schleife abhängig von äußerer Schleife. Anzahl der Durchläufe nimmt mit ansteigendem i ab (n – 1, n – 2, …).

$$\mathrm{n} - 1 + \mathrm{n} - 2 + \cdots + 2 + 1 = \frac{n * (n-1)}{2} = \frac{n^2 - n}{2}$$

$$\frac{n^2 - n}{2} \; ist\ in\ O(n^2)$$

Kubisch – $O(n^3)$

```
private static int[][] multiply(int[][] first, int[][] second) {
    int m = first.length;
    int n = second[0].length;
    int p = second.length;
    int[][] result = new int[m][n];
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < p; k++) {
                result[i][j] += first[i][k] * second[k][j];
            }
        }
    }
    return result;
}
```

Matrizenmultiplikation - Drei verschachtelte for-Schleifen
Worst-Case – zwei quadratische Matrizen multiplizieren
(dann sind m, n und p gleich groß)

Polynomiell allgemein – $O(n^k)$

Viele bisherige Klassen enthalten (z. B. linear, quadratisch, kubisch)

Bei größerem k

- Werden Algorithmen in $O(n^k)$ unbrauchbar bzw. sind nur mehr auf kleinen Eingabegrößen sinnvoll
- Daher ist eine feinere Unterteilung (linear, quadratisch, kubisch) bei kleineren Werten für k sinnvoll

Exponentiell – O(k^n)

```
private static void solveHanoi(int n, String a, String b, String c) {
    if (n > 0) {
        solveHanoi(n - 1, a, c, b);
        System.out.println(a + " --> " + c);
        solveHanoi(n - 1, b, a, c);
    }
}
```

$2^n - 1$ Ausgaben bei n Scheiben und daher in $O(2^n)$

Exponentiell – Türme von Hanoi

Beispiele für $n = 2$, $n = 3$ und $n = 4$

Achtung (wächst sehr schnell)

- Z. B. 1023 Ausgaben bei $n = 10$
- Z. B. 1.048.575 Ausgaben bei $n = 20$

```
Tower 1 --> Tower 2
Tower 1 --> Tower 3
Tower 2 --> Tower 3
```

```
Tower 1 --> Tower 3
Tower 1 --> Tower 2
Tower 3 --> Tower 2
Tower 1 --> Tower 3
Tower 2 --> Tower 1
Tower 2 --> Tower 3
Tower 1 --> Tower 3
```

```
Tower 1 --> Tower 2
Tower 1 --> Tower 3
Tower 2 --> Tower 3
Tower 1 --> Tower 2
Tower 3 --> Tower 1
Tower 3 --> Tower 2
Tower 1 --> Tower 2
Tower 1 --> Tower 3
Tower 2 --> Tower 3
Tower 2 --> Tower 1
Tower 3 --> Tower 1
Tower 2 --> Tower 3
Tower 1 --> Tower 2
Tower 1 --> Tower 3
Tower 2 --> Tower 3
```

Hinweise zur O-Notation

Beschreibt asymptotisches Verhalten

- Für kleine Eingabegrößen kaum aussagekräftig

Kleinste obere Schranke wählen

- Matrizenmultiplikation (eigentlich $O(n^3)$) ist z. B. auch in $O(n^{10})$
- Das hilft aber nicht viel bei der Auswahl von Algorithmen

Für große Eingabegrößen

- Algorithmus mit kleinerer oberer Schranke wählen
- Z. B. $O(n \log n)$ gegenüber $O(n^2)$ bevorzugen

Maximale Abschnittssumme (1)

```
private static int maxSum1(int[] input) {
    int maxSum = input[0];
    for (int left = 0; left < input.length; left++) {
        for (int right = left; right < input.length; right++) {
            int sum = 0;
            for (int k = left; k <= right; k++) {
                sum += input[k];
            }
            if (sum > maxSum) {
                maxSum = sum;
            }
        }
    }
    return maxSum;
}
```

Drei verschachtelte Schleifen!
Alle Obergrenzen von der Länge von input abhängig – $O(n^3)$

Maximale Abschnittssumme (2)

```
private static int maxSum2(int[] input) {
    int maxSum = input[0];
    for (int left = 0; left < input.length; left++) {
        int sum = 0;
        for (int right = left; right < input.length; right++) {
            sum += input[right];
            if (sum > maxSum) {
                maxSum = sum;
            }
        }
    }
    return maxSum;
}
```

Zwei verschachtelte Schleifen!
Alle Obergrenzen von der Länge von input abhängig – $O(n^2)$

Maximale Abschnittssumme (3)

```
private static int maxSum3(int[] input) {
    int maxSum = input[0], endSum = maxSum;
    for (int pos = 1; pos < input.length; pos++) {
        endSum = Math.max(endSum + input[pos], input[pos]);
        maxSum = Math.max(maxSum, endSum);
    }
    return maxSum;
}
```

Eine Schleife über alle Elemente des Arrays – $O(n)$
Hinweis: Die Laufzeit von `Math.max` ist in $O(1)$

Kann das noch verbessert werden?

```
private static int maxSum3(int[] input) {
    int maxSum = input[0], endSum = maxSum;
    for (int i = 1; i < input.length; i++) {
        endSum = Math.max(endSum + input[i], input[i]);
        maxSum = Math.max(maxSum, endSum);
    }
    return maxSum;
}

private static int maxSum4(int[] input) {
    int maxSum = input[0], endSum = maxSum;
    for (int i = 1; i < input.length; i++) {
        int help = endSum + input[i];
        endSum = help > input[i] ? help : input[i];
        if (endSum > maxSum) {
            maxSum = endSum;
        }
    }
    return maxSum;
}
```

Variante ohne Methodenaufrufe

Ergebnisse

Eingabegröße n	maxSum3 (sec)	maxSum4 (sec)	Faktor
20 000 000	0.023	0.014	1.64
40 000 000	0.036	0.021	1,71
80 000 000	0.070	0.042	1.66
160 000 000	0.139	0.076	1.82
320 000 000	0.267	0.155	1.72
640 000 000	0.529	0.296	1.78

Laufzeit **linear** für beide Implementierungen

- maxSum3 braucht länger als maxSum4
- Faktor (Zeit für maxSum3 / Zeit für maxSum4) schwankt aber nur um einen bestimmten Wert